A 2D Mobile-Robot Navigation Simulator: Grid Planning and Feedback Control Implemented from Scratch

[Your name] · Department of Automation · October 2026

Code: github.com/ASylviaofmine/robot-nav-sim

Abstract. This report describes a navigation stack for a planar mobile robot, written from scratch in Python with no robotics framework. A grid map is searched with A* under a Manhattan heuristic, and the resulting waypoints are followed by a proportional go-to-goal controller. Rather than asserting that the system works, every claim is measured: the planner is cross-checked against breadth-first search on five hand-built maps (identical path lengths in 16 of 16 cases), the effect of the heuristic is quantified (130 versus 250 node expansions on the same problem), and tracking accuracy is reported as cross-track error (0.090 m maximum, 0.013 m mean) alongside a sensitivity sweep over three controller parameters. Two failure modes encountered during development — heading wrap-around and waypoint skipping — are documented with their measured consequences.

1. Introduction

Given a map with obstacles, a start pose and a goal pose, a mobile robot must produce a collision-free route and then execute it. The problem decomposes into planning (a discrete search over the map) and control (a continuous feedback law that keeps the robot on the planned route). This project implements both, plus the visualization needed to inspect what happened. The constraint that shaped every decision was to avoid a robotics framework: a framework would have produced a working demo but no understanding, and it would have hidden the two failure modes reported in §4.3 — exactly the kind of defect that writing the loop by hand exposes.

2. System architecture

The code is organized as five modules with strictly one-way dependencies: `grid` (map representation and cell/world coordinate conversion) → `planner` (A*) → `simulation` (assembles one complete run) → `viewer` (Pygame window and GIF export). `controller` depends on nothing but NumPy: it accepts a list of waypoints and never imports the map, so replacing the planner with a sampling-based method would not touch it. A single entry point, `python main.py`, wires the modules together.

3. Methods

3.1 Planning: A on a four-connected grid*

A* expands nodes in order of `f(n) = g(n) + h(n)`, where `g` is the cost already paid and `h` estimates the remaining cost. With unit step costs and a Manhattan heuristic `h = |Δr| + |Δc|`, `h` never overestimates the true remaining steps — it is admissible — so the first time the goal is expanded, the path is optimal. The search is `O(N log N)` in the number of cells for the heap operations. Out-of-bounds neighbours are treated as walls, which removes boundary checks from the inner loop.

3.2 Control: proportional go-to-goal with heading wrapping

At each step the controller computes the bearing to the current waypoint, `θ* = atan2(Δy, Δx)`, the heading error `e = θ* − θ`, and applies $\theta \leftarrow \theta + K_p\, e\, \Delta t$ before advancing along the new heading. The error is first wrapped into `(−π, π]`. Without wrapping, the accumulated heading grows without bound and the correction saturates; the robot circles a waypoint indefinitely (measured: 100 m travelled, 4 of 17 waypoints reached). A waypoint is considered reached when the distance falls below a tolerance `tol`, at which point the index advances by exactly one.

4. Results

All figures below are produced by scripts in the repository and can be regenerated with a single command; runtimes are medians over repeated runs on one Windows machine and are indicative only.

Table 1. Planner behaviour on five 10×10 maps (A*, Manhattan heuristic). Every path length equals the optimum found by an independent breadth-first search.

Map	Path (cells)	Steps	Nodes expanded	Time (ms)
Open grid	15	14	63	0.390
Horizontal wall	17	16	78	0.411
Vertical wall	17	16	71	0.452
Serpentine maze	17	16	65	0.410
Goal sealed off	—	—	94	0.595

Heuristic choice. Table 2 isolates the heuristic by running the same code with the weight `w` in `h` varied: `w = 0` degenerates to Dijkstra, `w = 1` is A*. On a 20×20 map with 25 % random obstacles the Manhattan heuristic reduces the expanded set from 250 to 130 nodes with no loss of optimality; pushing `w` to 5 halves the work again but returns a 40-step path where 38 was available.



Figure 1. One complete run. Blue: the A* route. Red: the trajectory the controller actually drove. The two coincide to within 0.09 m, and no cell of the wall is entered.

Table 2. Effect of the heuristic weight on the same 20×20 random map (25 % walls).

w	Search	Path (steps)	Nodes expanded	Time (ms)
0	Dijkstra	38	250	1.410
1	A* (optimal)	38	130	0.682
2	Weighted A*	38	46	0.272
5	Near-greedy	40	45	0.331

Tracking accuracy. The closed loop reaches the goal in 32.00 s, travelling 16.00 m against a planned 16.00 m, with an end-point error of 0.039 m. Cross-track error, measured as the distance from each trajectory sample to the planned polyline, peaks at 0.090 m and averages 0.013 m; the largest deviations occur at the two 90° corners, where the robot cuts the turn.

Table 3 contains a result worth separating from the usual accuracy/speed trade-off. Between `tol` = 0.02 and 0.15 the maximum cross-track error does not move at all — it stays at 0.0895 m — while the end-point error grows by a factor of ten. The peak lateral deviation is therefore fixed by the corner geometry, not by the tolerance: the controller cuts each 90° turn by roughly the same amount however early it accepts a waypoint. What `tol` actually changes is how soon the robot turns towards the next waypoint, and that shows up as distance saved (16.13 → 15.50 m) rather than as lateral drift. Reading the two error columns together is what makes this visible; either one alone would have suggested a simple trade-off.

Table 3. Sensitivity to the waypoint tolerance `tol` (speed 0.5 m/s, $K_p = 5$). Loosening the tolerance buys time and distance but costs accuracy.

tol (m)	Time (s)	Distance (m)	End error (m)	Max cross-track (m)
0.02	32.25	16.13	0.015	0.0895
0.05	32.00	16.00	0.039	0.0895
0.15	31.00	15.50	0.144	0.0895
0.30	29.80	14.90	0.289	0.1596



Figure 2. Each controller parameter swept in isolation. Raising the heading gain from $K_p = 5$ to 12 cuts maximum cross-track error from 0.090 m to 0.023 m at essentially no time cost (32.00 → 31.75 s). Speed trades accuracy for time directly: 0.5 → 1.5 m/s shortens the run from 32.0 s to 11.2 s but triples the cross-track error to 0.283 m.

5. Limitations

- **Discrete planning.** A four-connected grid produces 90° corners and one-cell clearance; the robot footprint is ignored, so trajectories are not physically smooth.
- **No replanning, ideal state estimation.** The controller follows the route it was given and cannot detect obstacles that appear afterwards; pose is assumed known exactly, with no noise or sensor model.
- **Correctness depends on the tracker.** The path is certified collision-free only if the waypoint index advances one at a time. Advancing by two cuts a corner and enters the wall — measured, not hypothesised.
- **Single machine, single map family.** Runtimes come from one laptop; weighted terrain, larger maps and diagonal motion are untested.

6. Conclusion

A planner, a controller and a viewer, each written by hand and each measured, form a closed navigation loop whose behaviour is understood end to end. The measurements show that a Manhattan heuristic halves the search on the tested maps without sacrificing optimality, that proportional heading control tracks the planned route to within nine centimetres, and that both defects found along the way were invisible until the numbers were inspected. Natural next steps: a sampling-based planner on identical maps, and a true differential-drive model with wheel-level commands.

References

1. P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
2. S. M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006. (Ch. 2 on discrete planning; Ch. 8 on feedback control.)
3. Pygame and Pillow documentation — 2D rendering, event loop, and GIF composition.